

CS-600: Software Design and Development

7-1 Short Paper: Integration Requirements and Design Decisions

Malgorzata Debska

Southern New Hampshire University

Matthew Scuteri

June 1, 2026

Integration Requirements and Design Patterns for External Service Communication

Modern software applications rarely run in isolation. Most organizations depend on external services such as payment gateways, analytics platforms, cloud storage providers, and customer relationship management systems to support business operations. Effective integration between internal applications and external services is critical for ensuring reliable data exchange, system performance, and user satisfaction. For this analysis, I selected a subscription management application that integrates an external payment gateway through application programming interfaces (APIs). This integration allows customers to subscribe to services, process payments, and manage billing information securely.

Analysis of Integration Requirements

The subscription management application must exchange information with the payment gateway whenever a customer creates, updates, or cancels a subscription. Data flows from the internal application to the payment gateway when payment information is given. The payment gateway then confirms the transaction and returns a response showing whether the payment was successful or not. The internal application stores the transaction results and updates the customer's subscription status accordingly. Several dependencies exist within this integration. The subscription application depends on the payment gateway's API availability, response time, authentication mechanisms, and security requirements. If the external service becomes unavailable or experiences delays, the internal application may not be able to process customer

payments efficiently. Another dependency involves keeping compatibility with API updates released by the payment provider.

Potential challenges include network failures, communication delays, security vulnerabilities, and error handling. Since financial information is exchanged between systems, protecting sensitive customer data is essential. Additionally, system scalability must be considered because transaction volumes may increase as the business grows. Applications must be designed to accommodate higher workloads without reducing performance or reliability.

Recommended Integration Pattern

The integration pattern most suitable for this scenario is the API Gateway pattern combined with a Service Layer pattern. An API Gateway serves as a centralized entry point for communication between the subscription management application and the external payment service. Rather than allowing different components of the application to communicate directly with the payment gateway, requests are routed through the gateway, which manages authentication, request validation, logging, and traffic control. The Service Layer pattern further improves the design by separating business logic from communication logic. The application's subscription management functions are still independent from the payment processing implementation. This separation makes the system easier to support and update when external services change. These patterns support scalability because added services can be connected through the gateway without major modifications to the application architecture. Security is strengthened because authentication and authorization controls are managed centrally. Maintainability improves because developers can change integration components without affecting core business functions.

Addressing Integration Challenges

I selected the API Gateway and Service Layer patterns after evaluating the integration requirements, system dependencies, and potential risks. My analysis focused on finding a solution that would support secure communication while reducing coupling between the internal application and the external service. The API Gateway addresses security challenges by providing a centralized location for authentication, encryption, monitoring, and access control. It also improves reliability by handling retries, rate limiting, and error responses when communication problems occur. This reduces the impact of temporary service outages and network disruptions. The Service Layer pattern addresses maintainability challenges by isolating external service interactions from the application's business rules. If the payment provider changes its API or if the organization decides to switch providers, developers can change the service layer without redesigning the entire application. Together, these patterns create a flexible architecture that supports efficient communication while minimizing operational risks.

According to Richards and Ford (2020), loosely coupled architectures improve adaptability and make systems easier to support as business requirements evolve. Similarly, Fowler (2018) explains that separating application concerns through design patterns improves system organization and reduces long-term development complexity.

Preparing for Future Software Development

Understanding integration requirements and design patterns is an important skill for future software development projects. Modern applications often interact with cloud services, third-party APIs, and distributed systems. Analyzing data flow, dependencies, and technical risks before implementation, developers can make informed architectural decisions that improve

software quality. Knowledge of integration patterns also helps developers design systems that are scalable, secure, and maintainable. As software environments become increasingly interconnected, professionals must understand how to manage communication between systems while minimizing complexity and ensuring reliability. These skills support better decision-making throughout the software development lifecycle and contribute to the successful delivery of business solutions.

Conclusion

Integrating a subscription management application with a payment gateway requires careful consideration of data flow, dependencies, security requirements, and system reliability. The combination of the API Gateway and Service Layer patterns provides an effective solution that supports secure, scalable, and maintainable communication between systems. Evaluating integration requirements before implementation helps organizations reduce risk and create flexible architectures capable of adapting to future business needs.

References

Fowler, M. (2018). *Patterns of enterprise application architecture*. Addison-Wesley Professional.

Richards, M., & Ford, N. (2020). *Fundamentals of software architecture: An engineering approach*. O'Reilly Media.